



UNIVERSIDAD
CATÓLICA DE
TEMUCO

Proyecto Final: Matemática Discreta

Ruta óptima entre ciudades mediante grafos ponderados

Estudiante: Diego Fonseca Aguilera

Ramo: Matemática Discreta

Profesor: Elliott Mardones Arias

Introducción

Este informe tiene como objetivo modelar una red de transporte internacional con el uso de una teoría de grafos, con el fin de desarrollar una aplicación capaz de determinar la ruta óptima entre distintas ubicaciones. Para este propósito, se ha seleccionado una red compuesta por 15 ciudades europeas, reconocidas por su alta interconectividad.

El problema que solucionaremos al crear esta aplicación será encontrar el camino de menor costo entre una ciudad de origen y una ciudad de destino elegidas por el usuario. Para representar estas conexiones del mundo real, se ha definido como criterio para ponderar el peso de las aristas, la distancia terrestre aproximada en kilómetros entre dichas ciudades utilizando las principales rutas de transporte ferroviario y vial. Este criterio de distancia métrica se mantendrá de forma consistente a lo largo de todo el desarrollo matemático y computacional del proyecto

Modelamiento Matemático del Grafo

Para poder resolver el problema de la optimización de rutas, la red de transporte terrestre se modela mediante un grafo ponderado, definido matemáticamente como:

$$G = (V, E, w)$$

Conjuntos de vértices (V):

Los vértices del grafo representan las 15 ciudades internacionales seleccionadas para el estudio. utilizando la notación de conjuntos, el conjunto V se define así:

$$V = \{\text{Madrid, Barcelona, París, Londres, Bruselas, Ámsterdam, Berlín, Munich, Viena, Praga, Varsovia, Budapest, Zurich, Milan, Roma}\}$$

El orden del grafo, que corresponde a la cantidad total de vértices, es $n=|V|$ [citestart]=15.

Conjunto de aristas (E):

Las aristas del grafo representan las conexiones viales directas existentes en la realidad entre las ciudades, El conjunto E esta compuesto por pares no ordenados de vértices adyacentes, estresado formalmente mediante notación de conjuntos:

$$E = \{ \{\text{Madrid,Barcelona}\}, \{\text{Madrid,París}\}, \{\text{Barcelona,París}\}, \{\text{París,Londres}\}, \{\text{París,Bruselas}\}, \{\text{Londres,Bruselas}\}, \{\text{Bruselas,Ámsterdam}\}, \{\text{Ámsterdam,Berlín}\}, \{\text{París,Múnich}\}, \{\text{Múnich,Berlín}\}, \{\text{Múnich,Viena}\}, \{\text{Berlín,Praga}\}, \{\text{Praga,Viena}\}, \{\text{Viena,Budapest}\}, \{\text{Berlín,Varsovia}\}, \{\text{Varsovia,Praga}\}, \{\text{Múnich,Zúrich}\}, \{\text{Zúrich,Milán}\}, \{\text{Milán,Roma}\}, \{\text{París,Zúrich}\} \}$$

Función de peso (w):

La función de peso asigna un valor numérico real y positivo a cada una de las conexiones del sistema, representando la distancia física en kilómetros entre cada par de ciudades, esta función es:

$$w : E \rightarrow R^+$$

Los valores correspondientes a las ponderaciones de cada arista de detallan de esta manera:

$$\begin{aligned} w(\{\text{Madrid, Barcelona}\}) &= 600 \text{ km} \\ w(\{\text{Madrid, París}\}) &= 1050 \text{ km} \\ w(\{\text{Barcelona, París}\}) &= 830 \text{ km} \\ w(\{\text{París, Londres}\}) &= 340 \text{ km} \\ w(\{\text{París, Bruselas}\}) &= 260 \text{ km} \\ w(\{\text{Londres, Bruselas}\}) &= 320 \text{ km} \\ w(\{\text{Bruselas, Ámsterdam}\}) &= 170 \text{ km} \\ w(\{\text{Ámsterdam, Berlín}\}) &= 580 \text{ km} \\ w(\{\text{París, Múnich}\}) &= 680 \text{ km} \\ w(\{\text{Múnich, Berlín}\}) &= 500 \text{ km} \end{aligned}$$

$$\begin{aligned} w(\{\text{Múnich, Viena}\}) &= 350 \text{ km} \\ w(\{\text{Berlín, Praga}\}) &= 280 \text{ km} \\ w(\{\text{Praga, Viena}\}) &= 250 \text{ km} \\ w(\{\text{Viena, Budapest}\}) &= 210 \text{ km} \\ w(\{\text{Berlín, Varsovia}\}) &= 520 \text{ km} \\ w(\{\text{Varsovia, Praga}\}) &= 510 \text{ km} \\ w(\{\text{Múnich, Zúrich}\}) &= 240 \text{ km} \\ w(\{\text{Zúrich, Milán}\}) &= 240 \text{ km} \\ w(\{\text{Milán, Roma}\}) &= 480 \text{ km} \\ w(\{\text{París, Zúrich}\}) &= 490 \text{ km} \end{aligned}$$

Tipo de Grafo Utilizado: El modelo que implemente se clasifica estructuralmente bajo las siguientes características

1. Grafo no dirigido: Las conexiones son bidireccionales, lo que implica que el orden de los vértices en la arista no altera el sentido del viaje, matemáticamente se cumple que $uv=vu$
2. Grafo simple: El sistema carece estrictamente de lazos (aristas que conectan un vértice consigo mismo) y de aristas paralelas (múltiples conexiones entre un mismo par de ciudades)
3. Grafo Conexo: Existe cohesión estructural en la red, asegurando la disponibilidad de trayectorias entre cualquier par de elementos del conjunto de ciudades

Proposiciones Lógicas del grafo:

Proposición de conexidad (P1): El apunte establece que un grafo G es conexo si, para toda participación de su conjunto de vértices en dos subconjuntos no vacías X y Y , existe al menos una arista con un extremo en X y el otro en Y . entonces la proposición lógica es:

P1: Si el conjunto de vértices V se dividen en dos subconjuntos no vacíos (X e Y), entonces existe una arista en E que une a un vértice de X con un vértice de Y

Proposición de grafo simple (P2): El modelo corresponde a un grafo simple, lo que significa que no tiene lazos ni aristas paralelas. considerando que un lazo se define como una arista cuyos dos extremos corresponden al mismo vértice ($e = uu$), la proposición lógica que asegura la ausencia de lazos en nuestra red es:

P2: Si u es un vértice cualquiera perteneciente a V , entonces no existe ninguna arista en E de forma uu

Significado de la ruta óptima en el contexto del proyecto:

Bajo los conceptos de la asignatura, dos vértices son adyacentes si existe una arista que los conecta directamente un camino se define como una secuencia de vértices donde los elementos sucesivos son adyacentes, este modelo, esto representa una ruta de viaje continua a través de conexiones de tren reales.

Cada una de estas conexiones posee un peso (w) que equivale a la distancia en kilómetros entre ciudades, el costo total del camino suma de los pesos de las aristas que lo componen. por lo tanto, la ruta óptima es el camino específico entre la ciudad de origen y la de destino que resulta en el menor costo posible, garantizando el trayecto más corto dentro de la red.

Descripción general de algoritmo de Dijkstra

Para encontrar la ruta óptima en el grafo conexo y simple construido, se seleccionó el algoritmo de Dijkstra. Este procedimiento es un algoritmo que determina el camino más corto desde un único vértice de origen hacia todos los demás vértices del grafo, deteniendo su ejecución de manera óptima cuando se alcanza el nodo de destino seleccionado por el usuario

El algoritmo funciona manteniendo un registro de la distancia mínima estimada desde el origen hasta cada ciudad. Inicialmente estas distancias se configuran como infinitas excepto para la ciudad de origen, cuya distancia es cero, a medida que explora la red, el algoritmo selecciona el vértice no visitado con la menor distancia tentativa, examina todos sus vecinos adyacentes y actualiza sus costos si encuentra un trayecto más corto a través del nodo actual

Pasos Principales del algoritmo:

- **Inicialización:** Asignar a cada vértice $v \in V$ un valor de distancia tentativa. Para el nodo de origen s , la distancia es 0, para el resto de los vértices la distancia es infinita, se marca el nodo de origen como activo.
- **Selección de vértice:** identificar de entre todos los vértices que no han sido procesados como (no visitados), aquel que posea la menor distancia tentativa. establecer este vértice como el “nodo actual”
- **Actualización de Vecindades:** Para el nodo actual, examinar todos sus vecinos directamente conectado (osea los vértices adyacentes), calcular la distancia acumulada sumando la distancia del nodo actual con el peso (w) de la arista que lo une al vecino, si este valor es menor que la distancia tentativa previamente registrada para ese vecino, se actualiza la distancia con el nuevo valor menor.
- **Marcado:** una vez evaluados todos los vecinos adyacentes. Al marcar el nodo actual como “visitado”, un nodo visitado no vuelve a ser evaluado.
- **Condición de término:** repetir los pasos 2,3 y 4 hasta que el nodo de destino sea marcado como visitado o hasta que todos los nodos accesibles hayan sido procesados.

Justificación de la elección y comparación con alguna alternativa:

La selección del algoritmo Dijkstra frente a otras alternativas, como el algoritmo de Bellman-Ford, se fundamenta estrictamente en las características de nuestro problema:

- **Pertenencia del tipo de pesos:** El algoritmo de Dijkstra exige como condición que todas las funciones de peso asignen valores reales positivos ($w: E \rightarrow \mathbb{R}^+$). Dado que nuestro criterio de ponderación es la distancia física en kilómetros entre ciudades, está garantizando de forma lógica que ninguna arista poseerá un costo negativo.
- **Eficiencia computacional:** el algoritmo de Bellman Ford es una alternativa capaz de resolver caminos mínimos incluso en presencia de aristas con pesos negativos, sin embargo debido a esa flexibilidad su complejidad temporal es bastante mayor, requiriendo procesar de forma reiterada todas las conexiones del grafo. al tener la certeza matemática que todas las distancias de las vías son positivas, la utilización de bellman-ford resulta ineficiente además de innecesario exclusivamente para este modelo en específico

Por lo que la elección de Dijkstra asegura la resolución exacta del problema de la ruta óptima con el menor consumo de recursos de cómputo posible, adaptándose de manera ideal a la interfaz gráfica y visualización requeridas por el proyecto.

Implementación computacional: Para el desarrollo de la aplicación utilice el lenguaje Python trabajando dentro de un entorno virtual aislado (.venv) para garantizar que no hayan problemas en las dependencias al compartir el archivo con usted

Estructura General y librerías utilizadas: El código fuente está organizado de forma modular para separar la lógica matemática del diseño visual para lograr esto se utilizaron las siguientes librerías principales:

NetworkX y Matplotlib: Utilizadas en conjunto para renderizar la representación visual del grafo en pantalla

CustomTkinter: Empleada para el desarrollo de la interfaz gráfica de usuario (GUI), permitiendo la interacción mediante menús desplegables y botones

Estructura de datos para la representación del grafo:

Para optimizar el rendimiento en Python, esta matriz teórica se implementó utilizando un diccionario de diccionarios. Las claves principales corresponden a las ciudades y sus valores son sub diccionarios que contienen las ciudades vecinas junto con las distancia en kilómetros:

```
1 grafo_ciudades = {
2     'Madrid': {'Barcelona': 600, 'Paris': 1050},
3     'Barcelona': {'Madrid': 600, 'Paris': 830},
4     'Paris': {'Madrid': 1050, 'Barcelona': 830, 'Londres': 340, 'Bruselas': 260, 'München': 680, 'Zürich': 490},
5     'Londres': {'Paris': 340, 'Bruselas': 320},
6     'Bruselas': {'Paris': 260, 'Londres': 320, 'Ámsterdam': 170},
7     'Ámsterdam': {'Bruselas': 170, 'Berlín': 580},
8     'Berlín': {'Ámsterdam': 580, 'München': 500, 'Praga': 280, 'Varsovia': 520},
9     'München': {'Paris': 680, 'Berlín': 500, 'Viena': 350, 'Zürich': 240},
10    'Viena': {'München': 350, 'Praga': 250, 'Budapest': 210},
11    'Praga': {'Berlín': 280, 'Viena': 250, 'Varsovia': 510},
12    'Varsovia': {'Berlín': 520, 'Praga': 510},
13    'Budapest': {'Viena': 210},
14    'Zürich': {'München': 240, 'Milán': 240, 'Paris': 490},
15    'Milán': {'Zürich': 240, 'Roma': 480},
16    'Roma': {'Milán': 480}
17 }
```

Algoritmo de Camino Mínimo (Código Fuente):

La lógica central de la aplicación recae en la función que ejecuta el algoritmo de Dijkstra. El siguiente fragmento de código se ve la implementación central de la búsqueda de ruta óptima:

Explicación Técnica: Este bloque de código inicializa las distancias de todos los vértices como infinito a excepción del origen. Utiliza una cola de prioridad (heapq) para seleccionar eficientemente el nodo con la menor distancia tentativa (cumpliendo con la metodología teórica). Luego, evalúa los vértices adyacentes actualizando los pesos mínimos y registrando el nodo de procedencia. Finalmente el código ejecuta un bucle iterativo de reconstrucción (while actual is not None) que retrocede desde el nodo de destino hacia el origen utilizando el registro previo, devolviendo así la secuencia exacta de la ruta óptima y su costo total estimado.

```
1 import heapq
2
3 def calcular_dijkstra(grafo, origen, destino):
4     distancias = {nodo: float('infinity') for nodo in grafo}
5     distancias[origen] = 0
6     camino_previo = {nodo: None for nodo in grafo}
7     cola_prioridad = [(0, origen)]
8
9     while cola_prioridad:
10        distancia_actual, nodo_actual = heapq.heappop(cola_prioridad)
11
12        if nodo_actual == destino:
13            break
14        if distancia_actual > distancias[nodo_actual]:
15            continue
16
17        for vecino, peso in grafo[nodo_actual].items():
18            distancia_tentativa = distancia_actual + peso
19            if distancia_tentativa < distancias[vecino]:
20                distancias[vecino] = distancia_tentativa
21                camino_previo[vecino] = nodo_actual
22                heapq.heappush(cola_prioridad, (distancia_tentativa, vecino))
23
24    ruta = []
25    actual = destino
26    while actual is not None:
27        ruta.insert(0, actual)
28        actual = camino_previo[actual]
29
30    if distancias[destino] == float('infinity'):
31        return [], 0
32    return ruta, distancias[destino]
```

El funcionamiento de interfaz Gráfica La interfaz gráfica fue diseñada para cumplir con los requisitos de la rúbrica:

Panel de selección: contiene Controles para que el usuario elija el nodo de origen y el nodo de destino de forma intuitiva, junto con un botón para ejecutar el cálculo

Panel de resultados: un área de texto que muestra de forma secuencial la lista de ciudades que componen el recorrido calculado y el costo total en kilómetros

Área de visualización: un lienzo donde se renderiza el grafo completo la aplicación dibuja los vértices como puntos y las conexiones reales como líneas al ejecutar el algoritmo, la secuencia de arista que conforman la ruta óptima se redibuja utilizando un color contrastante y un mayor grosor para distinguirse claramente del resto de la red

Resultados y Pruebas de funcionamiento:

Para validar la correcta implementación del algoritmo y el modelo matemático se ejecutaron diversas pruebas en la interfaz gráfica. A continuación se detallarán tres escenarios de búsqueda que demuestra la capacidad de la aplicación

Prueba 1: Recorrido de extremo a extremo

- Ciudad de origen: Madrid
- Ciudad de destino Varsovia
- Ruta óptima obtenida: Madrid, Paris, Bruselas, Ámsterdam, Berlín, Varsovia
- costo total estimado: 2580 km
- Análisis: El algoritmo descarto rutas hacia el centro de Europa al ver que la interconexión por los países bajo y el norte de alemania acumulaba una distancia menor

Prueba 2: Recorrido de distancia Media

- Ciudad de origen: Roma
- Ciudad de destino: Praga
- Ruta óptima obtenida: Roma, Milán, Zurich, Munich, Viena, Praga
- Costo total estimado: 1560 km
- Análisis: El sistema funciono correctamente conectando italia con europa central a través de suiza y austria respetando las conexiones directas del grafo

Prueba 3: Recorrido Corto

- Ciudad de origen: Londres
- Ciudad de destino: Munich
- Ruta óptima obtenida: Londres, parís, munich
- costo total estimado: 1020 km
- Análisis: Aunque londres está cerca de bruselas, el algoritmo detectó que el trayecto directo vía parís es más corto en kilómetros totales, demostrando la eficacia de la suma secuencial de pesos

Conclusión

El proyecto permitió integrar de forma práctica los fundamentos de la matemática discreta y la teoría de grafos mediante el desarrollo de un programa

Se logró comprender que la abstracción matemática es esencial antes de escribir cualquier línea de código. La transformación de una matriz teórica a una estructura de datos real facilitó la aplicación de algoritmos de optimización como Dijkstra. Uno de los principales retos fue la representación visual del grafo especialmente la distribución espacial de los nodos de la interfaz para que el diagrama resultante mantuviera una similitud comprensible con un mapa geográfico real.

Pensando en algunas mejoras futuras en vez de lo estático de distancia se podría reemplazar por tiempo de viaje o costo del pasaje y incluso conectar con un sistema de API en tiempo real para obtener datos ferroviarios dinámicos brindando múltiples opciones de optimización para el usuario

Link al GitHub

<https://github.com/sub-dll/proyecto-matematica-discreta>